COL7378: Principles of Autonomous Systems
Semester II, 2025-26

Markov Decision Processes

Rohan Paul

Outline

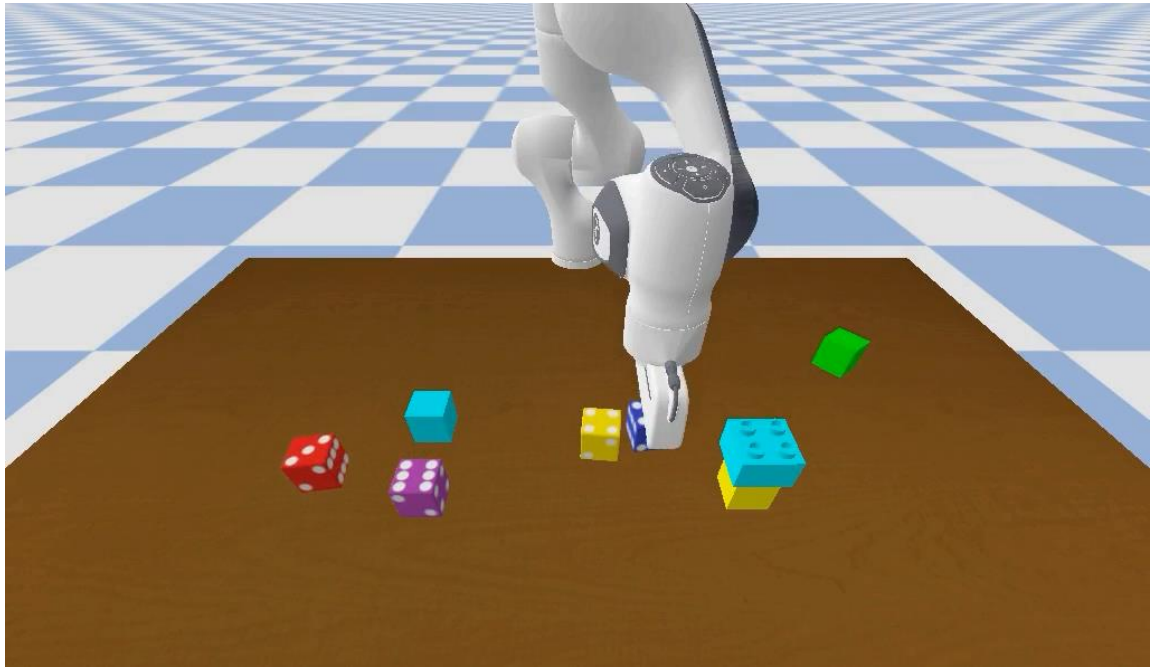
- Last Class
  - Motion Planning
- This Class
  - Sequential decision-making under uncertainty
    - Markov Decision Processes
- Reference Material
  - Primary reference are the lecture notes. For basic background refer to AIMA Classical Planning Ch. 17 (Sec 17.1 - 17.3). Other reference is Sutton and Barto, Reinforcement Learning Ch 3.

Acknowledgements

These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Nicholas Roy, Wolfram Burgard, Dieter Fox, Sebastian Thrun, Siddharth Srinivasa, Dan Klein, Pieter Abbeel, Max Likhachev and others.

Action Uncertainty

Problem: Actions may not have their desired effects. Errors, alternative outcomes, unmodeled dynamics ….
Solution: Need to model this behaviour using probabilities expressed as the transition model.



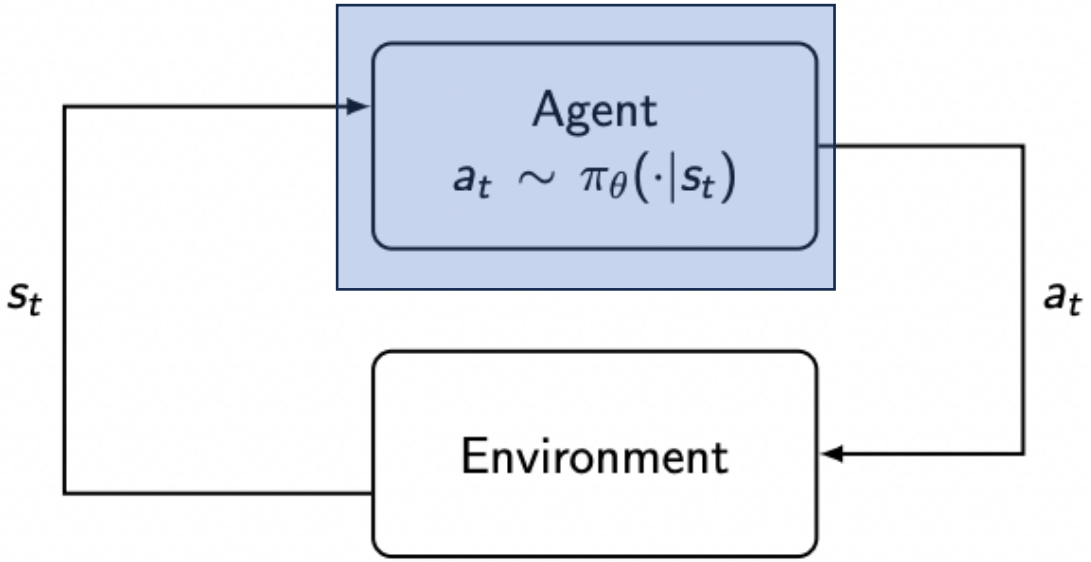
Namasivayam et al. ICRA ‘23, IROS ‘24

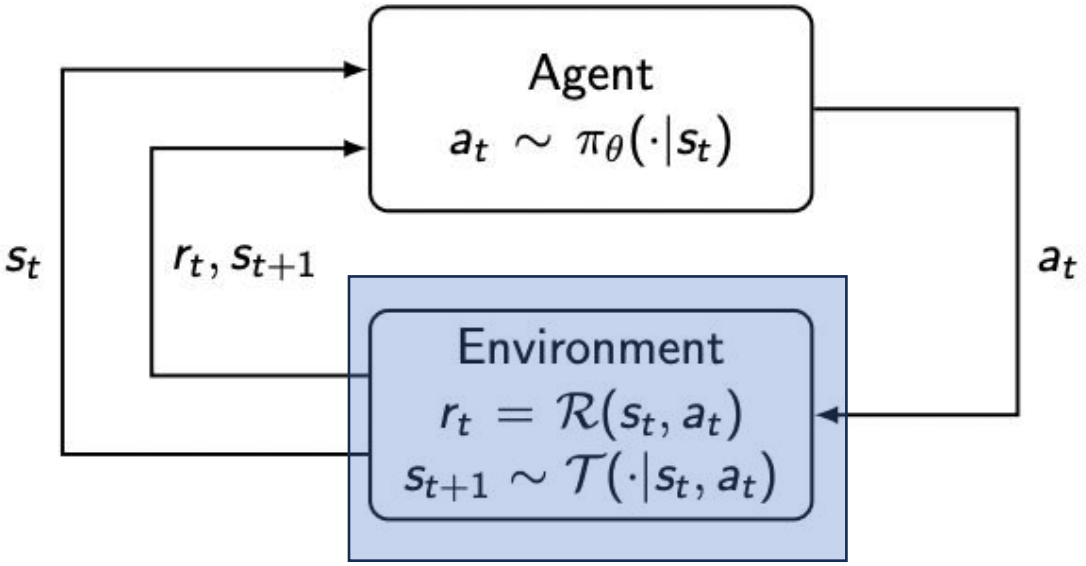
Field experiment where a robot is asked to move to an observation point.

We need some formulation to model autonomous systems and then a way to determine optimal behaviour.

Modeling the Agent-Environment Interface

To start, we need a representation. An agent in a Markov Decision Processes (MDPs) is a great starting point.





Agent learns and executes a policy. The policy is its behaviour. Its goal is to have the best policy for its objective.

The environment gives the agent a reward. It also transitions to the next state, not under the agent's control.

Markov Decision Processes: Components

The agent lives in a world that is a Markov Decision Process. It acts as per a policy. The reward model captures the goal it is provided to achieve.

Infinite-horizon, discounted MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \gamma \rangle$

- $\mathcal{S}$ Set of states
- $\mathcal{A}$ Set of actions
- $\mathcal{R}$ Reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$
- $\mathcal{T}$ Transition function $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$
- γ Discount factor $\gamma \in [0, 1)$

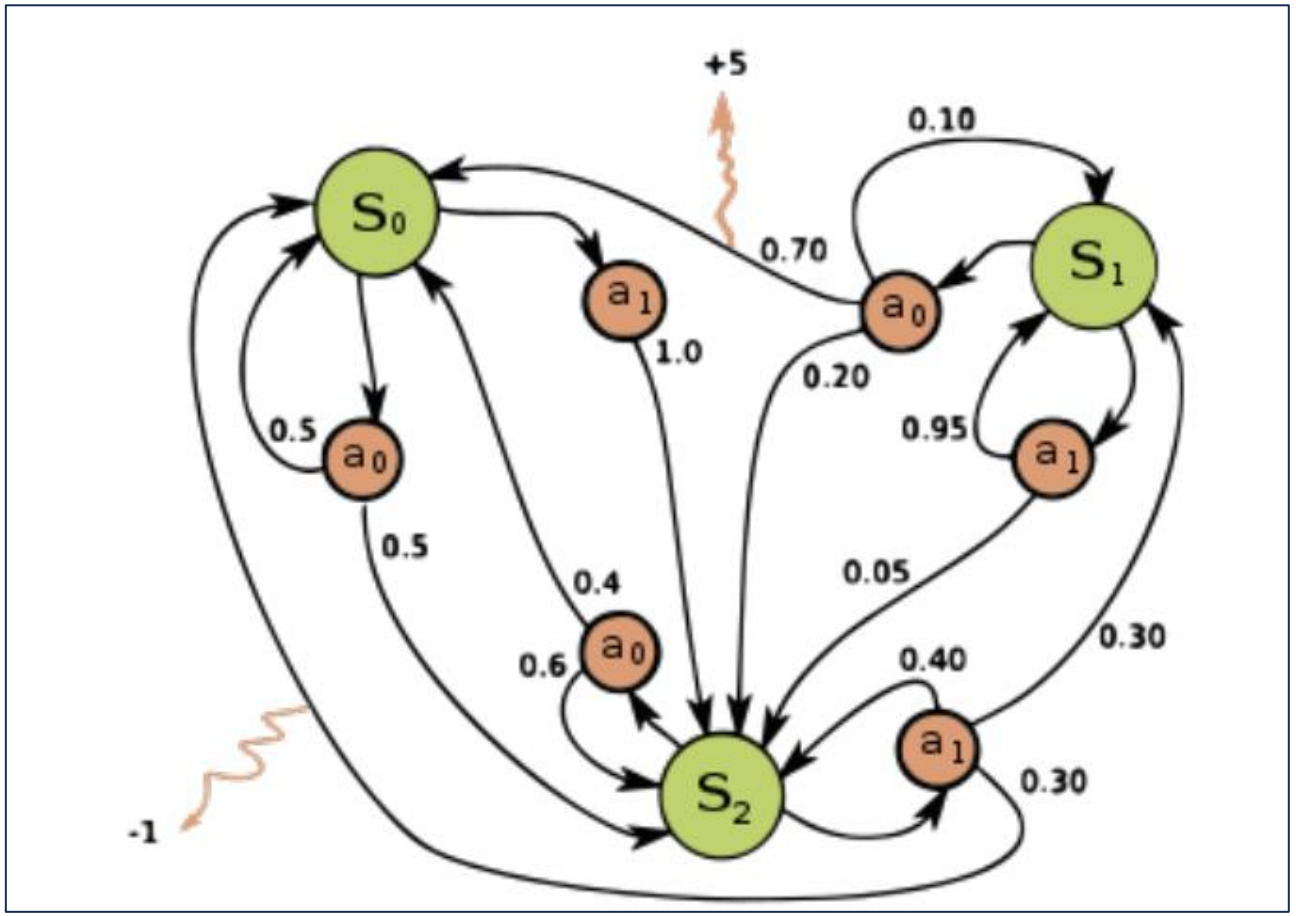
In MDP, these R and T are known

Behavior is encoded via a stationary, stochastic policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$

How do we assess the performance of a given policy π?

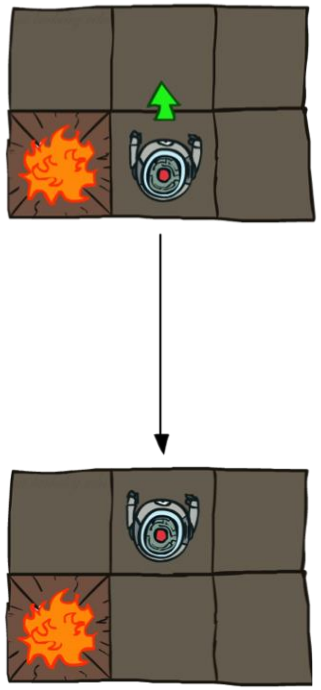
Agent is situated and acts “continually” in the MDP

- The agent lives in a world which is an MDP. It acts as per a policy.
- Moves in the environment transitioning states, taking actions and collecting rewards.
- At each point, looks at the state it is in, takes the action and reaches a new state based on the uncertainty in the interaction.

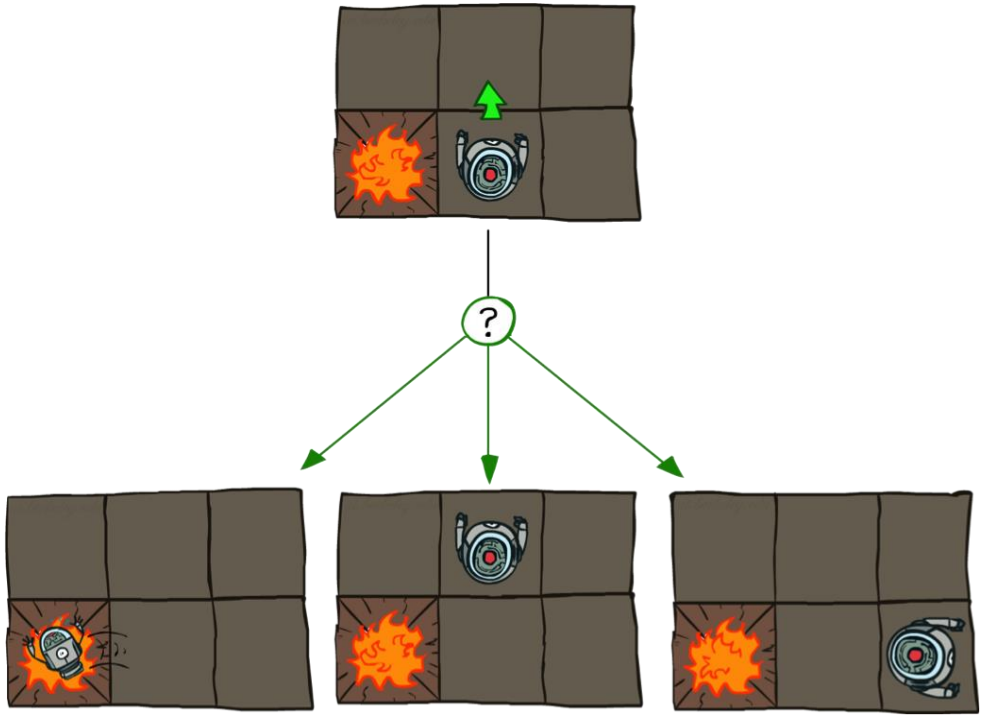


Consequences of Action Uncertainty

Deterministic Grid World



Stochastic Grid World

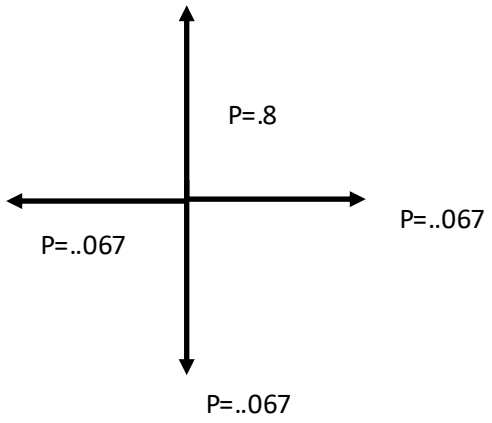


If the environment was *deterministic*, then we would just need an optimal plan (a sequence of actions) from start to the goal. If there is *non-determinism*, we are not sure where we will land up, hence need a policy that prescribes actions from each state.

Policies are Representing Behaviour

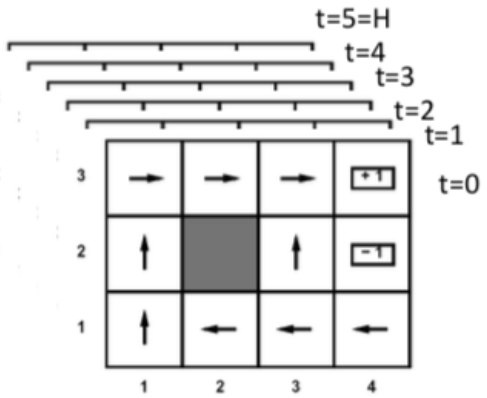
- Previously considered the deterministic case.
  - Actions (up, down, left, right) are deterministic, and each action incurs cost 1.
  - Deterministic means transition function is $T : S \times A \mapsto S$
  - Once the plan was computed, we can simply execute it.
- Now, consider the case where action outcomes are stochastic:
  - Transition function $T : S \times A \times S \mapsto [0, 1]$, $\sum_{s_j} T(\cdot, \cdot, s_j) = 1$.
  - Transition function for $(3, 2)$ and $(3, 1)$ is terminal; once reached, the agent cannot leave those states
  - Reward is $-.1$ everywhere except the terminal states, which have reward $+1$ and -1.
  - In this example, with probability .8 action has the "intended" outcome, and with some uniform probability (0.067) the agent ends up in one of the other feasible 4-connected states.
  - If the transition is into an obstacle or outside the grid, the agent's state does not change.

-.1	-.1	-.1	+1
-.1		-.1	-1
-.1	-.1	-.1	-.1



Plans Vs. Policies

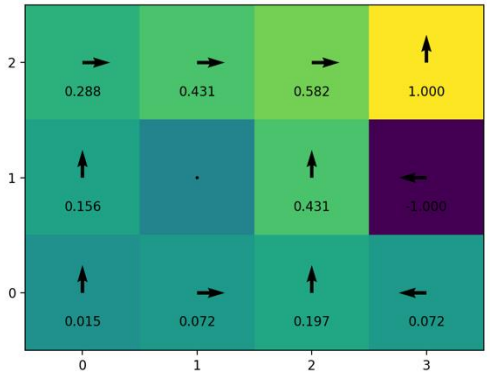
- Deterministic single-agent search problems
  - we wanted an optimal plan, or sequence of actions, from start to a goal
- For MDPs, we want an optimal policy $\pi^*: S \rightarrow A$
  - A policy π gives an action for each state
  - The agent arrives at a state and looks up the action according to the policy.



At any time, the agent will land up in a state.

- Which action to take in that state?
- Take the action prescribed by the policy.

- **Will any policy work?**
  - **No. We want an optimal policy is one that maximizes the expected utility if followed**

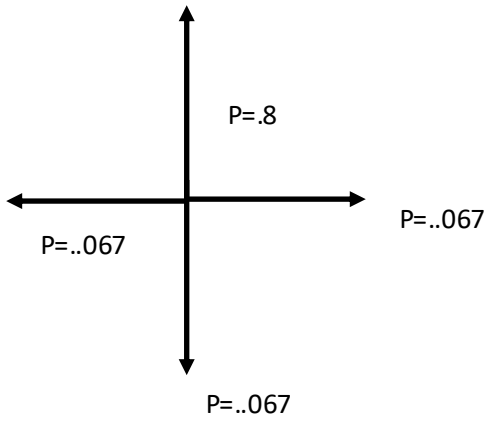


Example of a policy (arrows). The colors indicate the value of a state given the policy (see Value functions in a later slide).

Formulating and MDP Model

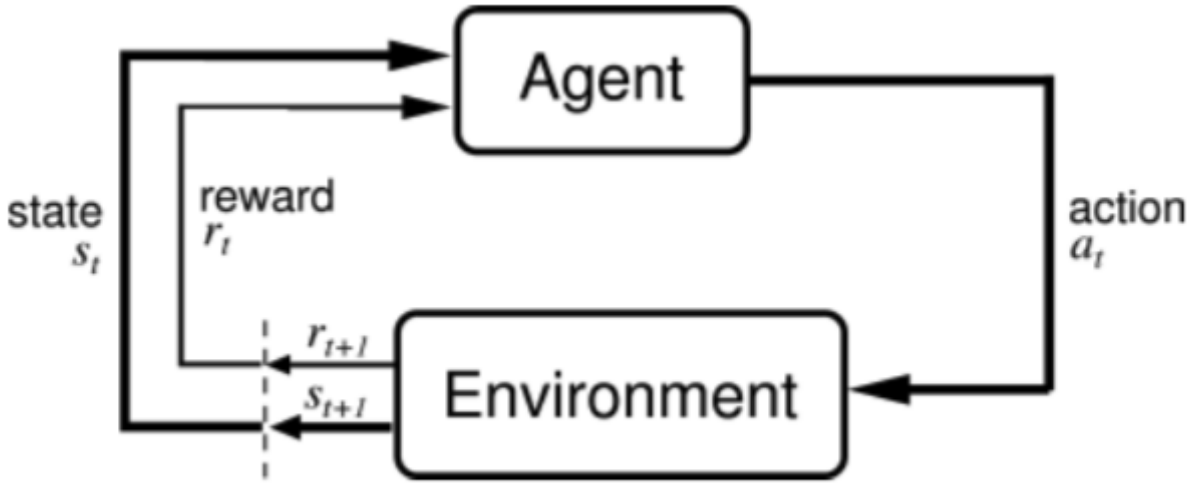
- Recall that the input to any solver is a model M.
- The MDP is defined as a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma, s^0)$
  - Finite set of states, $\mathcal{S} = \{s^0, \ldots, s^n\}$
  - Finite set of actions, $\mathcal{A} = \{a^0, \ldots, a^m\}$
  - State transition function $T(s^i, a^j, s^k)$ such that $T : S \times A \times S \mapsto [0, 1]$
  - Reward for each state-action-state transition $R(s^i, a^j, s^k)$ such that $R : S \times A \times S \mapsto \mathcal{R}$
  - Discount factor $\gamma \in [0, 1]$
  - Initial state s_0.
- Why is it called "Markov"?
  - Because the state dynamics depends *only* on the current state and action. $s_{t+1} \perp\!\!\!\perp s_{t-1} | s_t, a_t$.
- Called a decision process
  - Because we choose the actions

-.1	-.1	-.1	+1
-.1		-.1	-1
-.1	-.1	-.1	-.1



What objective does a policy optimize?

- MDP is a model for the agent
  - Our goal is to determine good behaviour for the agent.
- Solving an MDP to arrive at an optimal policy
  - Find a policy that maximizes the future expected reward.
  - That is, find a prescription of actions from each state such that the future expected reward is maximized.
  - The state is assumed known (we will relax that later). Right now, uncertainty is only in the actions.



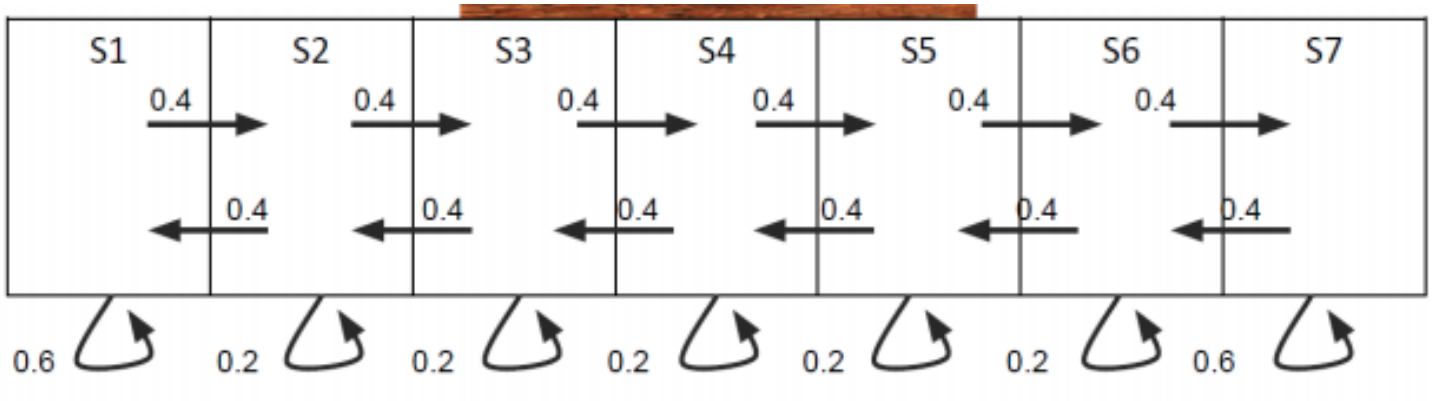
$$\pi^* = \arg\max_{\pi} \mathrm{E}[\sum_{t=0}^{H} \gamma^t R_t(S_t, A_t, S_{t+1})|\pi]$$

MDP Example: Mars Rover

States and rewards
(encode the agent's goal)

S1	S2	S3	S4	S5	S6	S7
Okay Field Site R=+1	R=0	R=0	R=0	R=0	R=0	Fantastic Field Site R=+10

Transition function.



Actions: Left or Right.
Policy: prescription of actions to states.

Assigning Rewards to Sequences

- Additive utility: $U([r_0, r_1, r_2, \ldots]) = r_0 + r_1 + r_2 + \cdots$
- Discounted utility: $U([r_0, r_1, r_2, \ldots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \cdots$

Discounting appears to be a good model for both animal and human preferences over time.

With discounted rewards, the utility of an infinite sequence is finite.

$$U([r_0, \ldots r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\text{max}}/(1-\gamma)$$

Notation: U() and V() are both used to denote value functions in literature.

Gamma inherently tells us how much further in the future we will account for.

Key Quantities

The value (utility) of a state s:

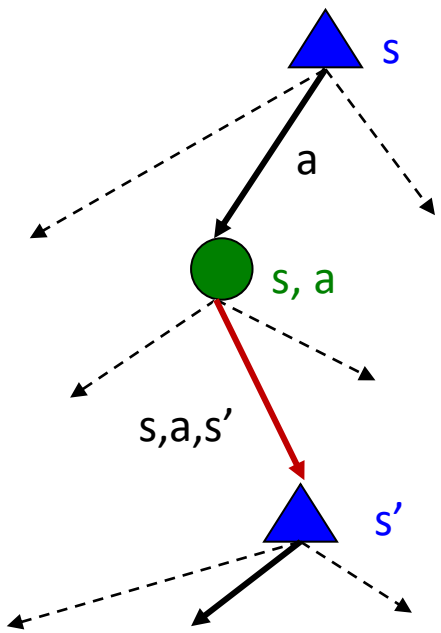
$V^*(s)$ = expected utility starting in s and acting optimally

The value (utility) of a q-state (s,a):

$Q^*(s,a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally

The optimal policy:

$\pi^*(s)$ = optimal action from state s



s is a *state*

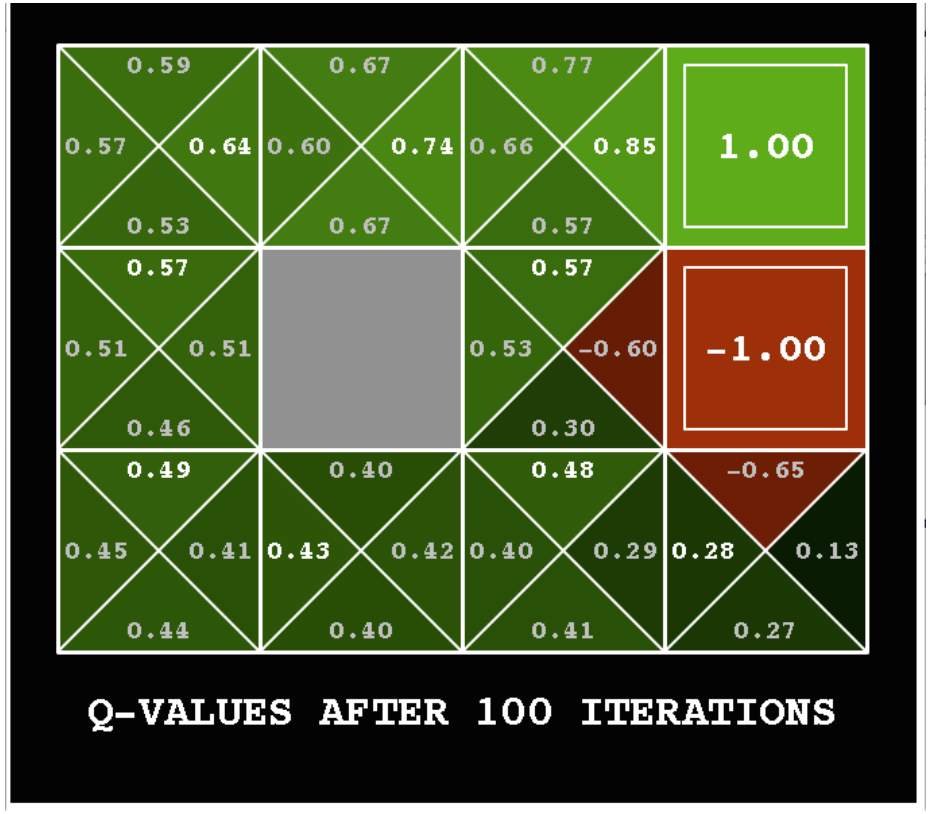
(s, a) is a *q-state*

(s,a,s') is a *transition*

Value Function for the Grid World

Value (utility) of states V(s) for all states

Value (utility) of a q-states Q(s,a) for all states and all actions at each state.



Noise = 0.2
Discount = 0.9
Living reward = 0

Bellman Equations

- Definition of **"optimal"** utility via a simple **one-step** lookahead relationship amongst optimal values.

Optimal value function for a state s maximizes the Q-values of the state s and applicable actions a.

$$V^*(s) = \max_a Q^*(s,a)$$

Q value function: how can we express it? Take the action a on state s and then act optimally.

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Recursive way to write the value function (we have seen this definition before).

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

- The utility of a state is the **immediate reward** for that state plus the **expected discounted utility of the next state** assuming that the agent is acting **optimally**.
- Note: The value function has some structure as they are modeling max. rewards that one can obtain from that state. The Bellman equations express that connection.

Value Iteration

- Core Idea
  - Calculate the optimal value function for each state. Then use the optimal value function to extract the optimal policy.
- Bellman Equations
  - The utility of a state is the immediate reward for that state plus the expected discounted utility of the next state assuming that the agent is acting optimally.

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

What is the intuition?

- Value of a state is not arbitrary, it is influenced by the neighbors.
- The Bellman equation encode this relationship.
- Methods to compute the policy exploit this relationship.

Value Iteration

Now we are not committing to any policy.
We assume the agent is free to choose the best action.
Here we take max because:
The agent will pick whichever action gives highest future return.

This answers:
"What is the best possible return achievable from this state?"

- Bellman equations ***characterize*** the optimal values:

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

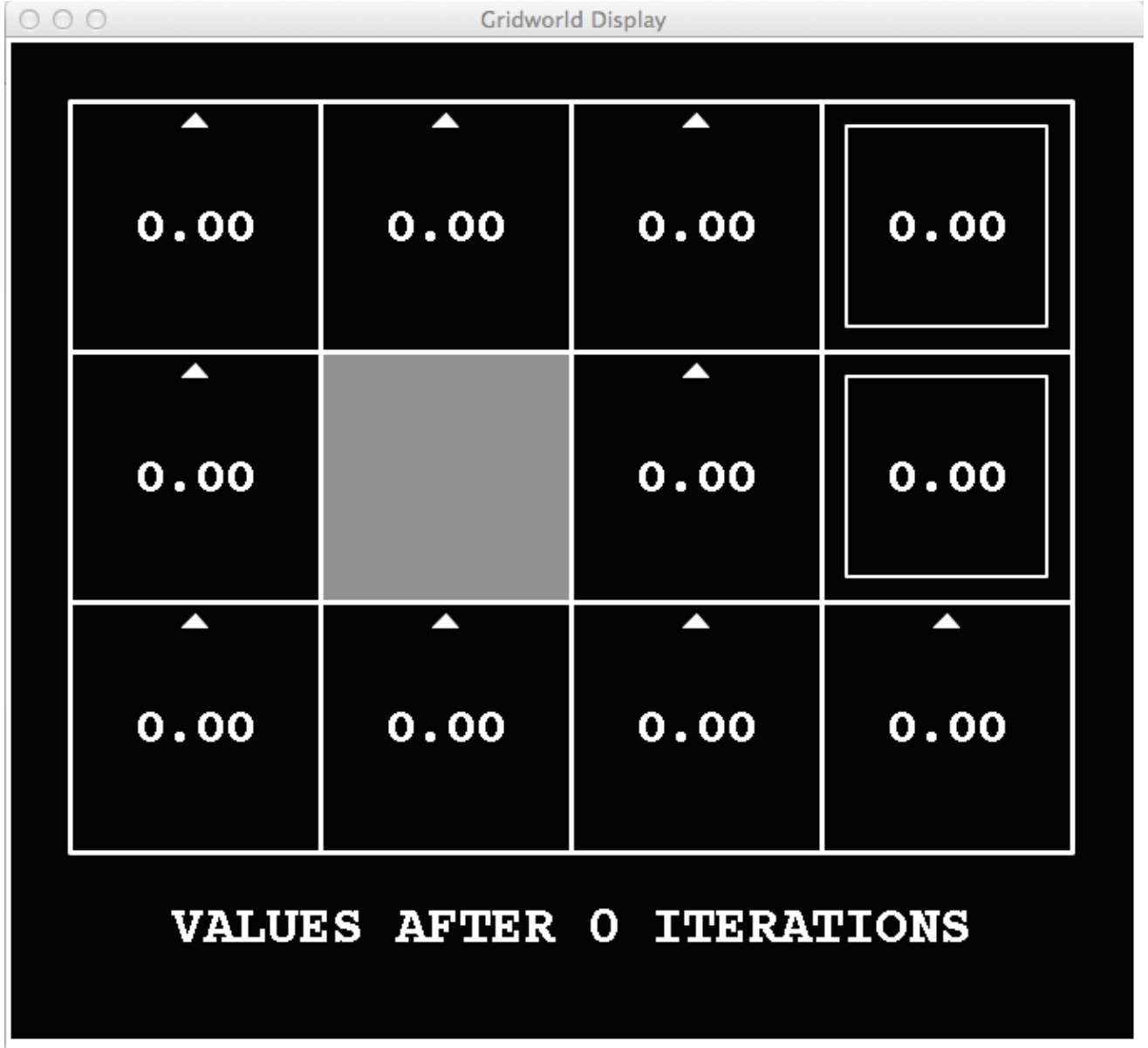
- Value iteration ***computes*** them with a fixed-point iteration also called the Bellman update/backup:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s')\right]$$

- In essence, value iteration provides the optimal policy for a finite horizon problem of length one, then two-step, then three step, …… and so on.
- Equivalently, one step of policy evaluation and one step of policy improvement.

Value Iteration

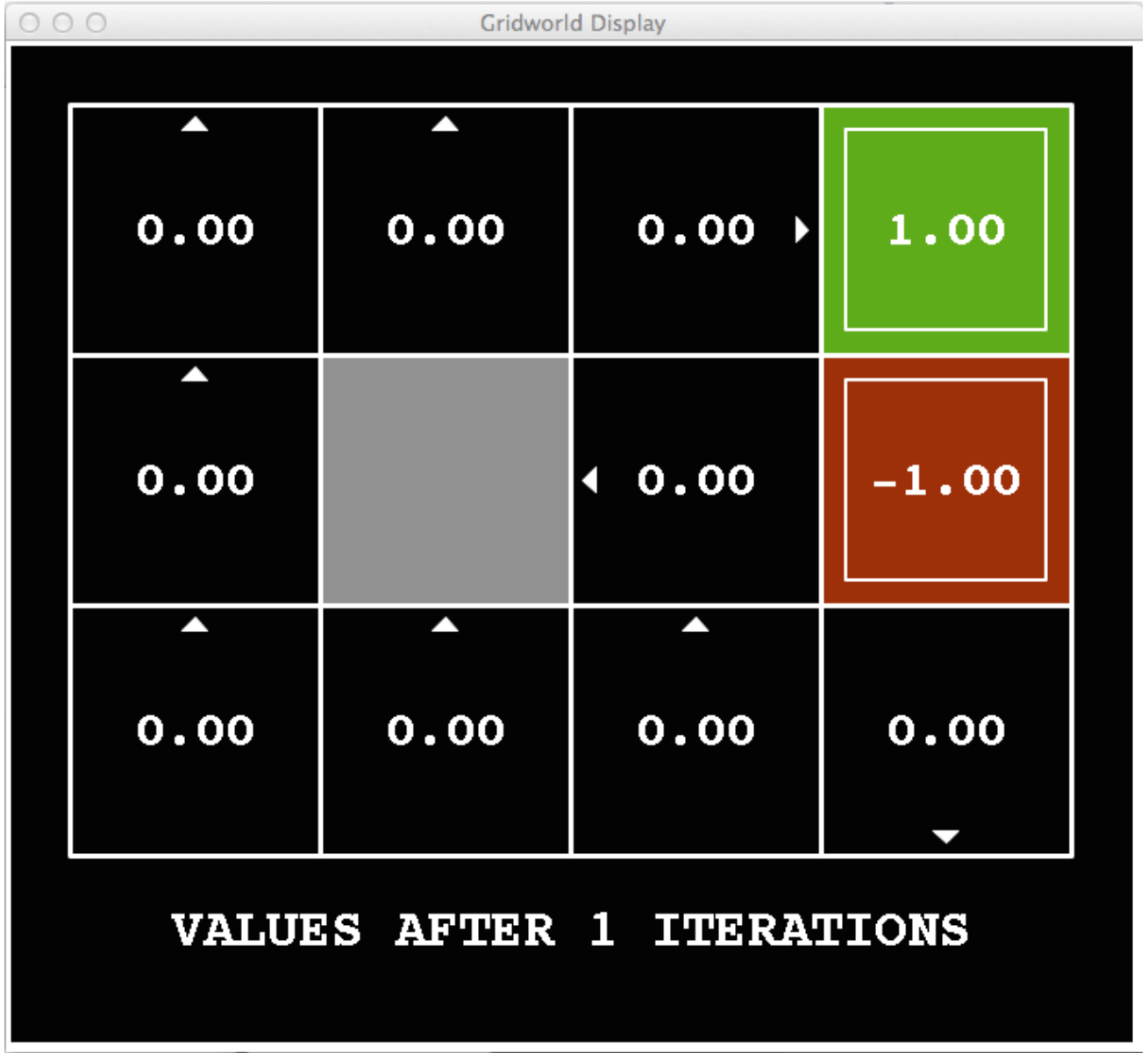
k=0



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

k=1



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

k=2

Noise = 0.2
Discount = 0.9
Living reward = 0

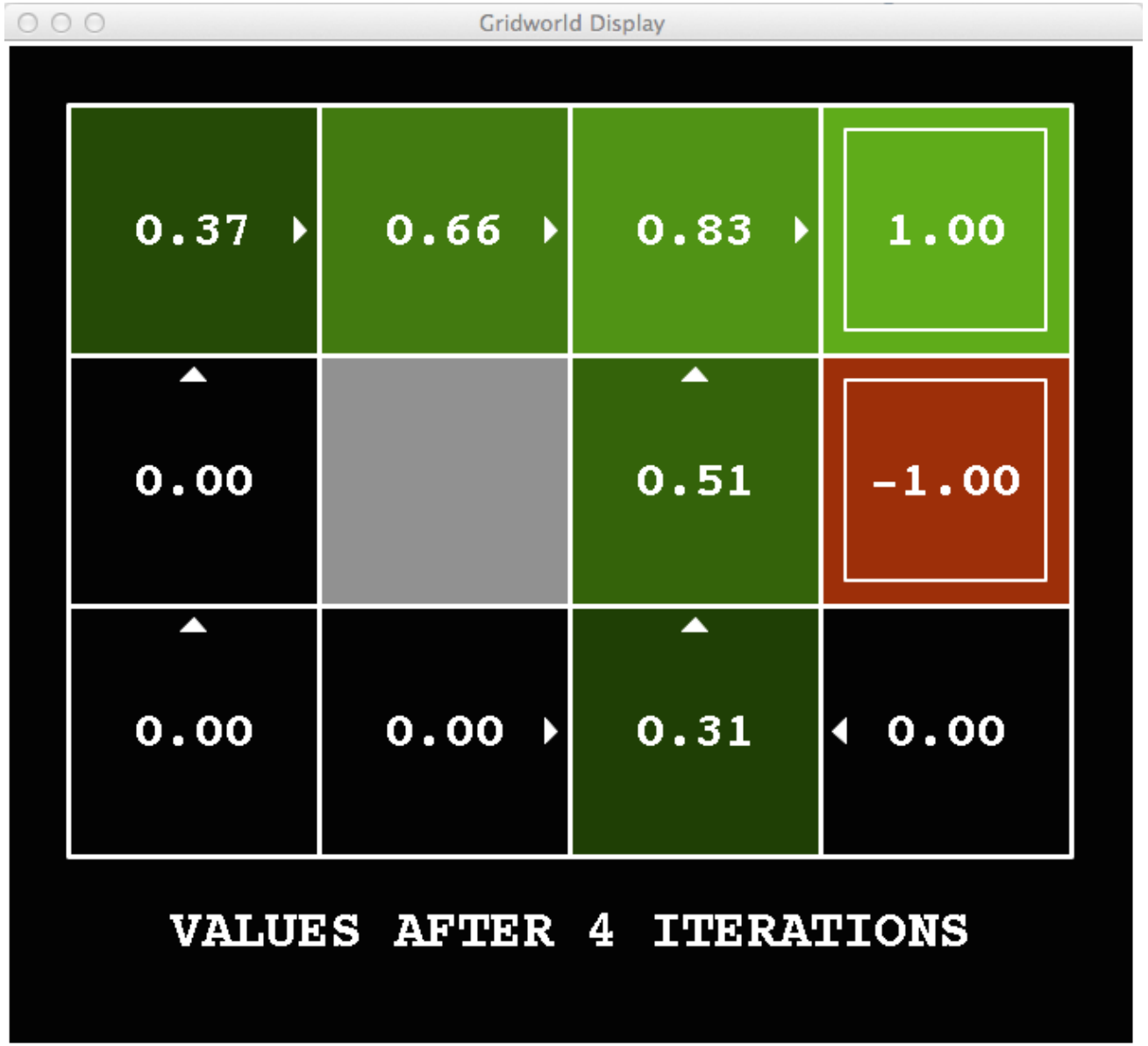
Value Iteration

k=3

Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

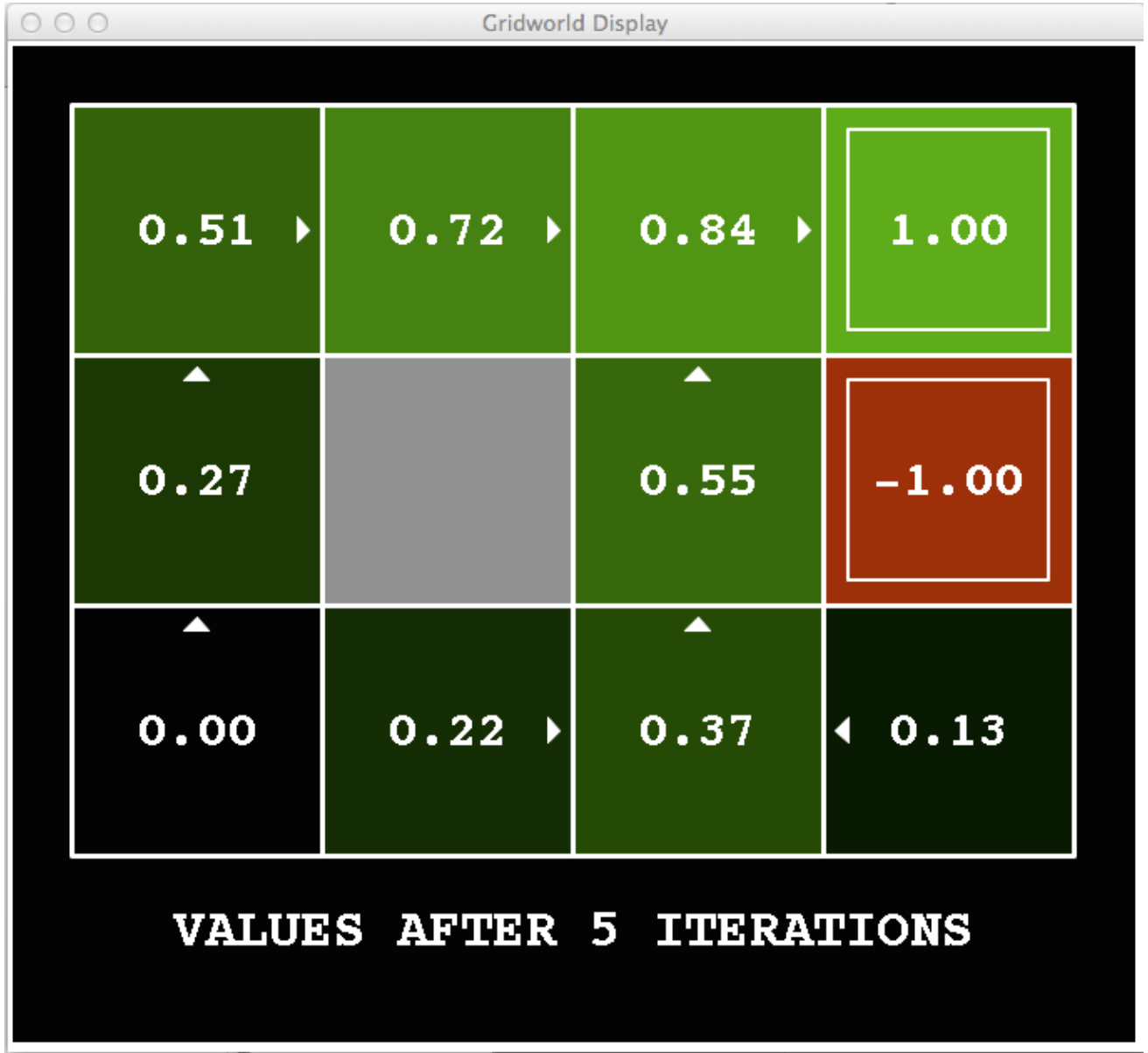
k=4



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

k=5



Noise = 0.2
Discount = 0.9
Living reward = 0

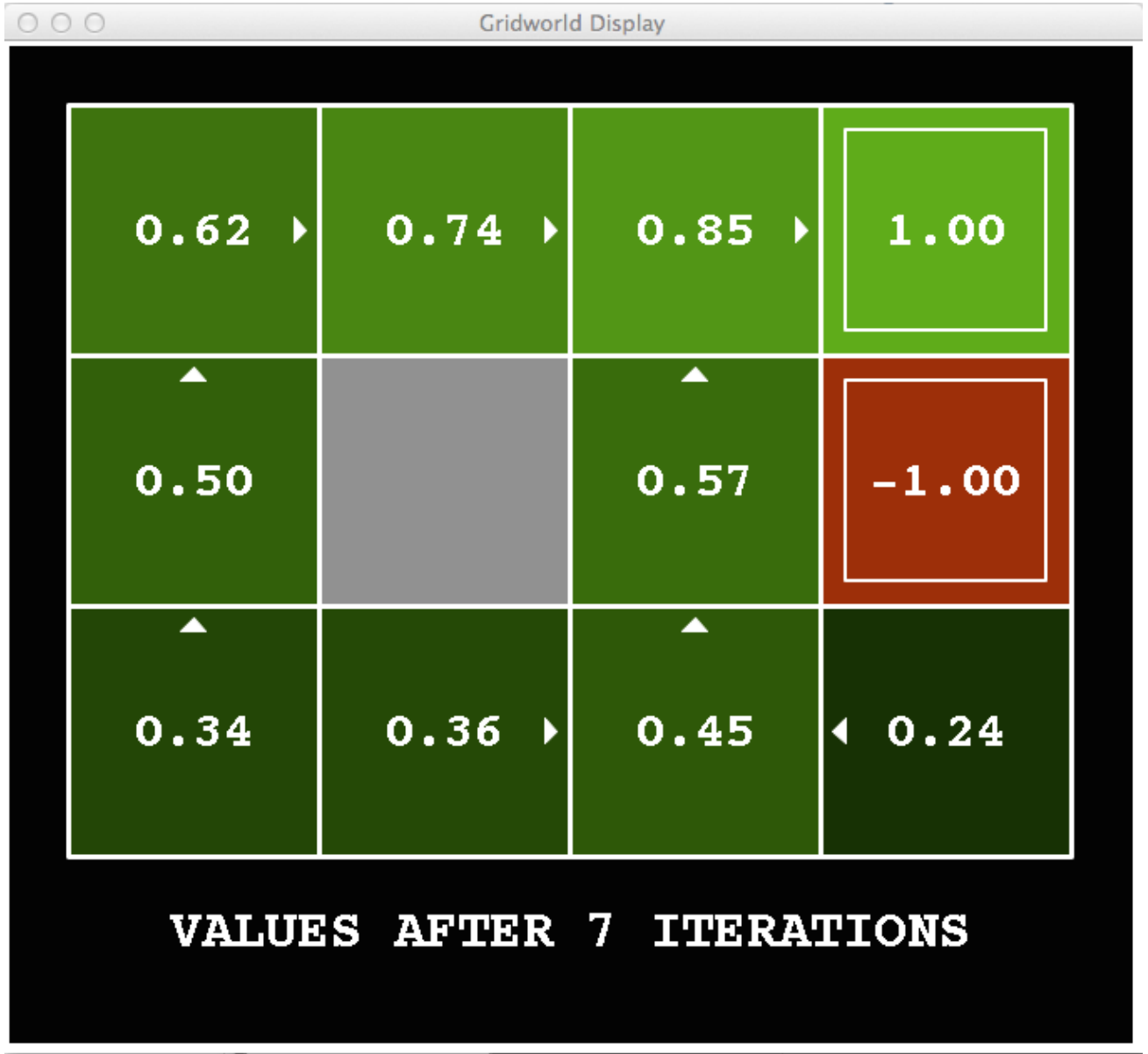
Value Iteration

k=6

Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

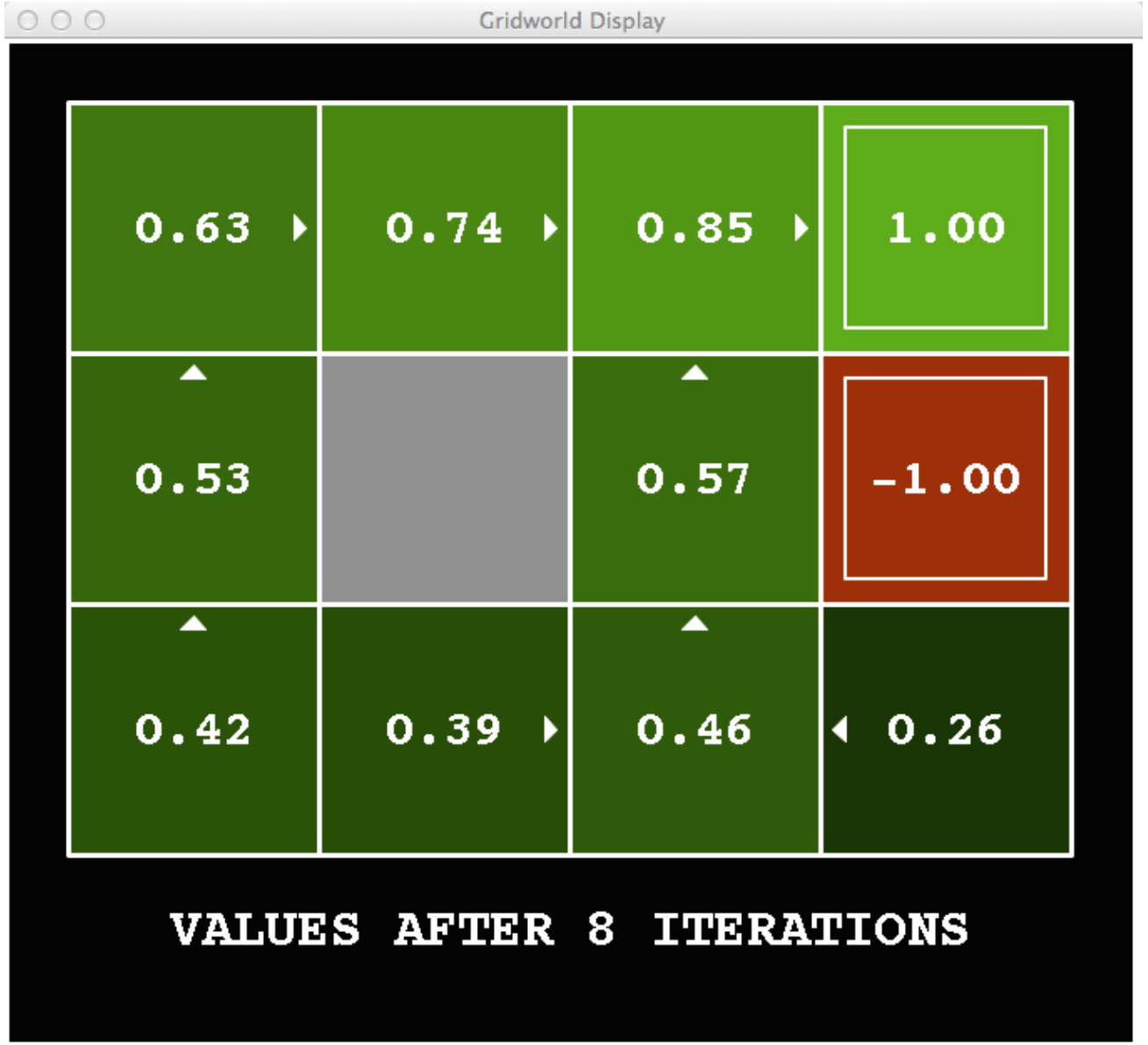
k=7



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

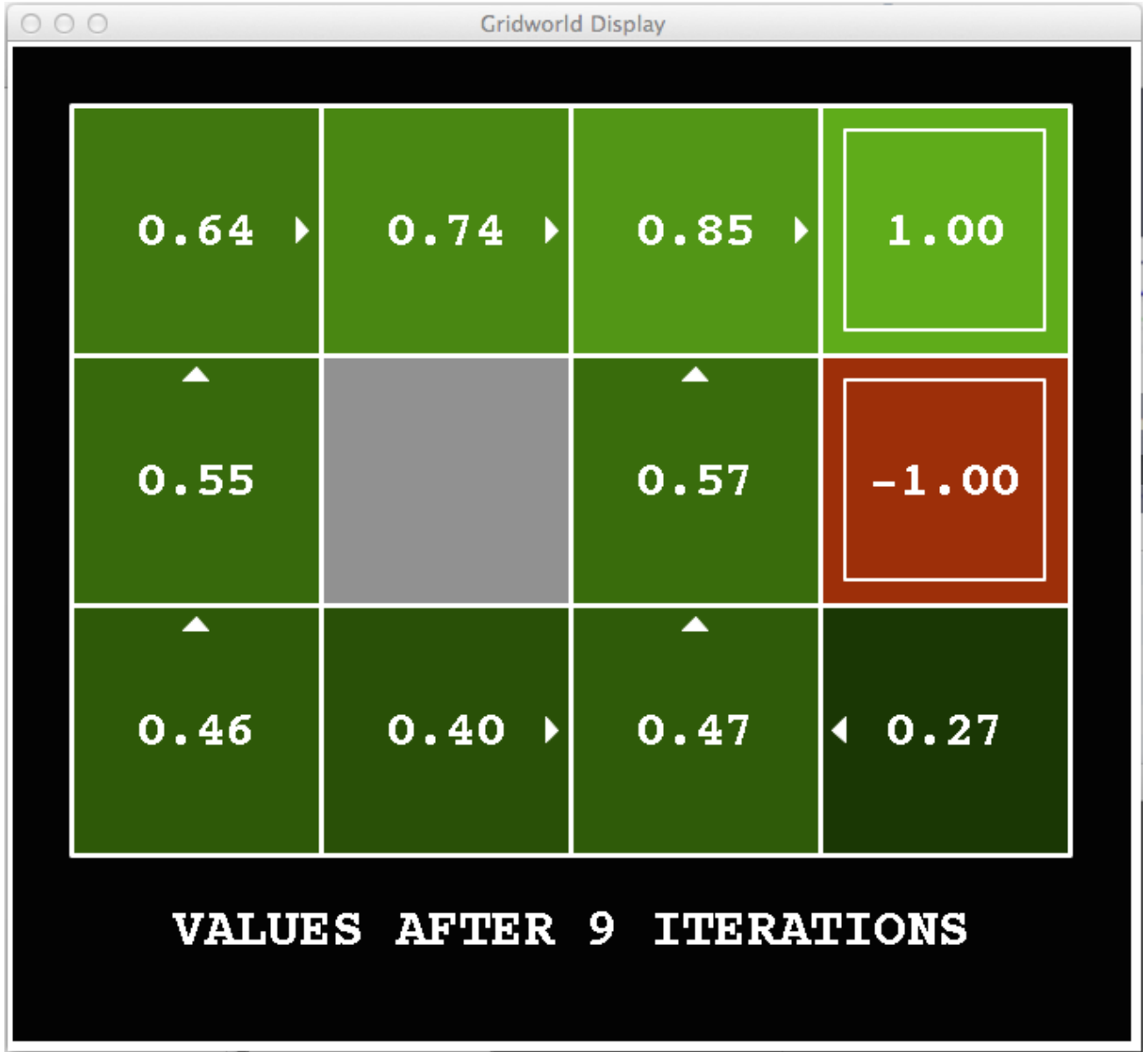
k=8



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

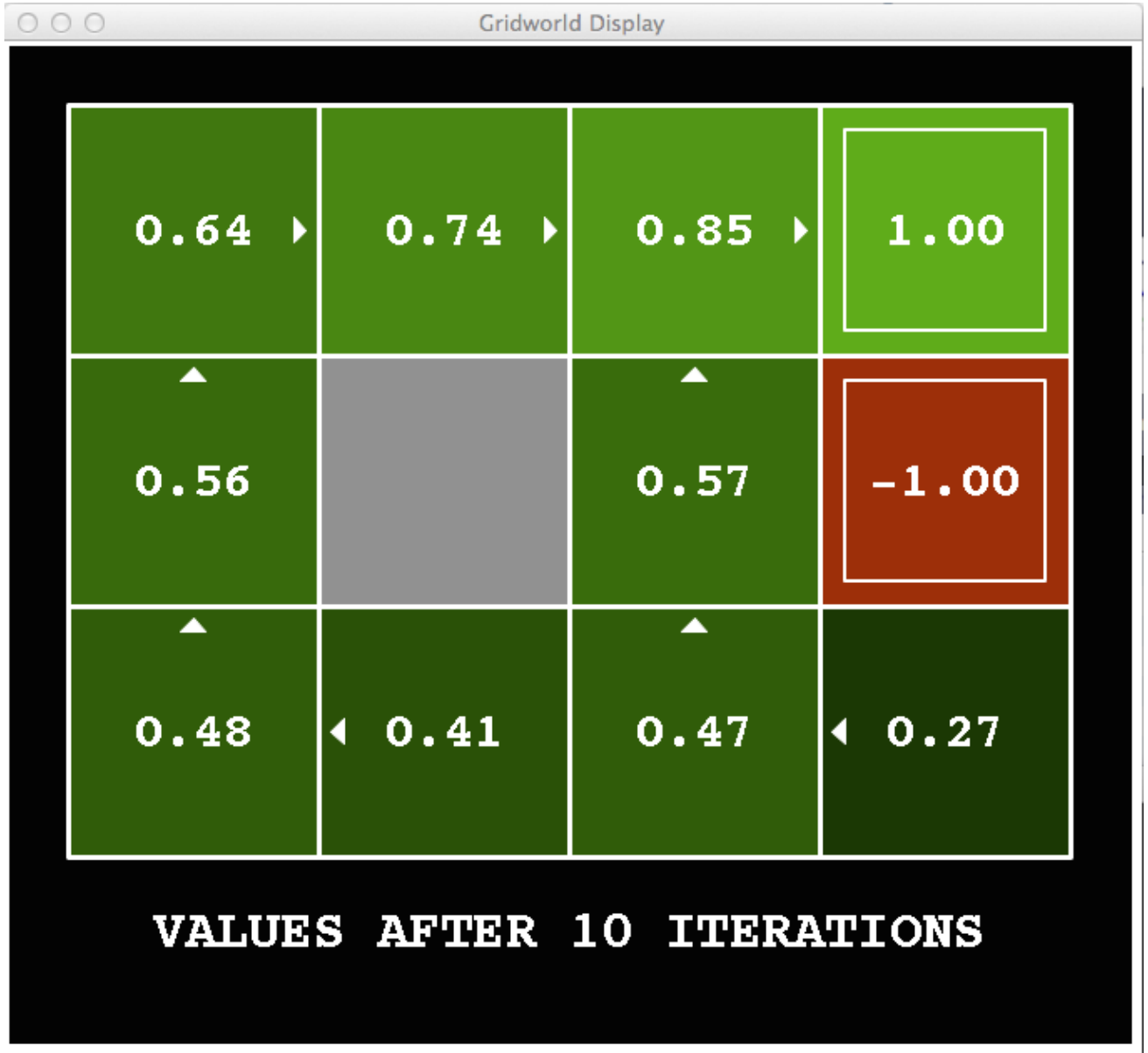
k=9



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

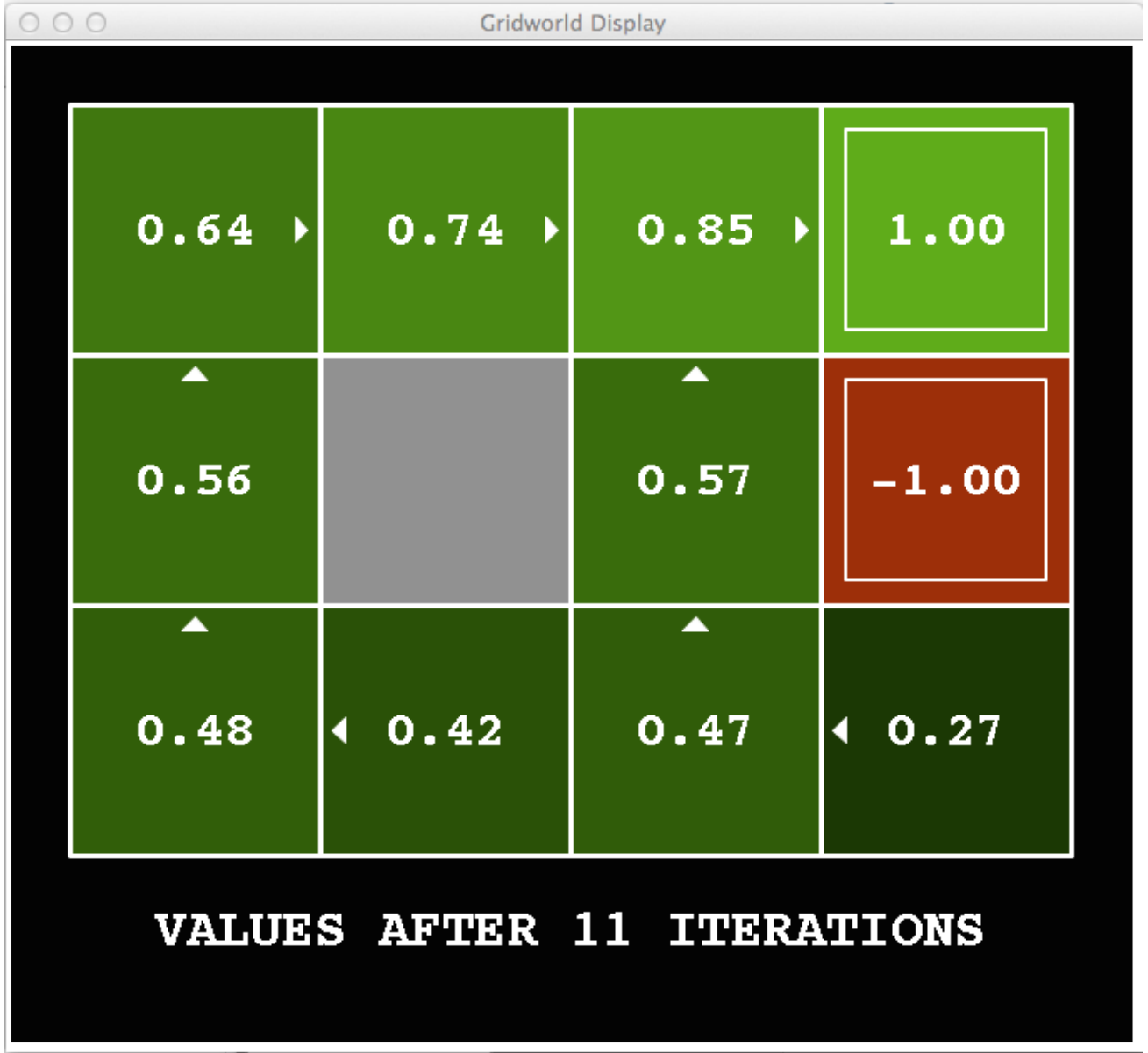
k=10



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

k=11



Noise = 0.2
Discount = 0.9
Living reward = 0

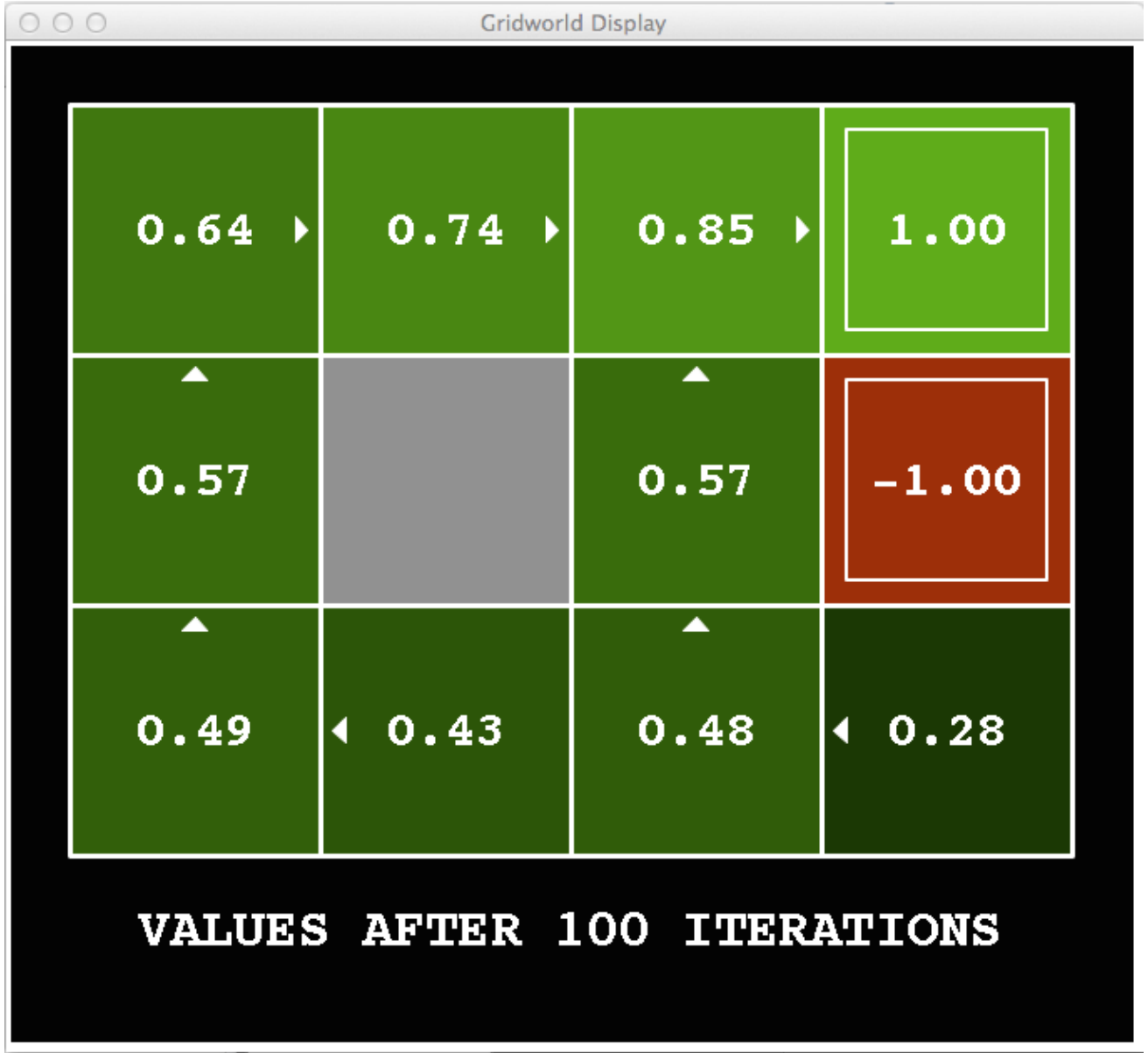
Value Iteration

k=12

Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

k=100



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

Sutton and Barto.

Stopping Criteria

Assuming and infinite horizon MDP with a discount factor less then a relationship between successive difference between value functions and w.r.t to the optimal can be shown.

$$\text{if} \quad ||U_{i+1} - U_i|| < \epsilon(1-\gamma)/\gamma \quad \text{then} \quad ||U_{i+1} - U|| < \epsilon$$

The relationship leads to a stopping criteria for value iteration.

function VALUE-ITERATION(mdp, ϵ) **returns** a utility function
  inputs: mdp, an MDP with states S, actions $A(s)$, transition model $P(s' \mid s, a)$,
    rewards $R(s)$, discount γ
    ϵ, the maximum error allowed in the utility of any state
  local variables: U, U', vectors of utilities for states in S, initially zero
    δ, the maximum change in the utility of any state in an iteration

  repeat
    $U \leftarrow U'$; $\delta \leftarrow 0$
    for each state s **in** S **do**
      $U'[s] \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' \mid s, a)\, U[s']$
      if $|U'[s] - U[s]| > \delta$ **then** $\delta \leftarrow |U'[s] - U[s]|$
  until $\delta < \epsilon(1-\gamma)/\gamma$
  return U

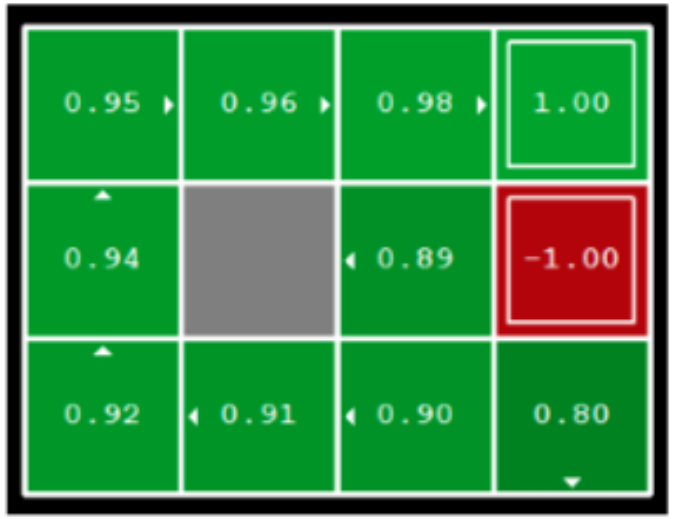
AIMA Ch. 17. Sec. 17.2

How to extract the policy?

- **Assume that we have the optimal values V*(s)**
  - After applying the value iteration algorithm
- **Policy Extraction**
  - Extract the policy implied by the computed value function by (using 1-step look ahead).

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Once we have the values computed, how to get the policy (arrows)?



Asynchronous Value Iteration (Prioritized Sweeping)

- Value iteration so far assumes all states in V_t are updated using values from prior value function V_{t-1}.
  - Requires keeping two functions
- Alternate approach is to keep a single value function estimate $\hat{V}$ and update states in order

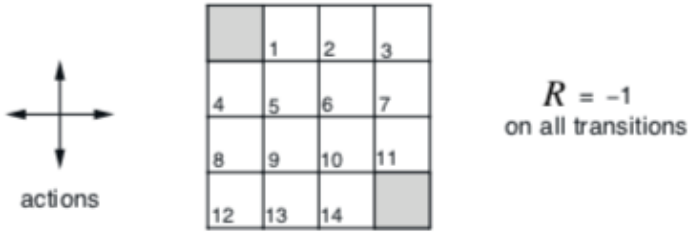
$$\hat{V}(s) = R(s) + \gamma \max_a \sum_s p(s'|s,a)V(s')$$

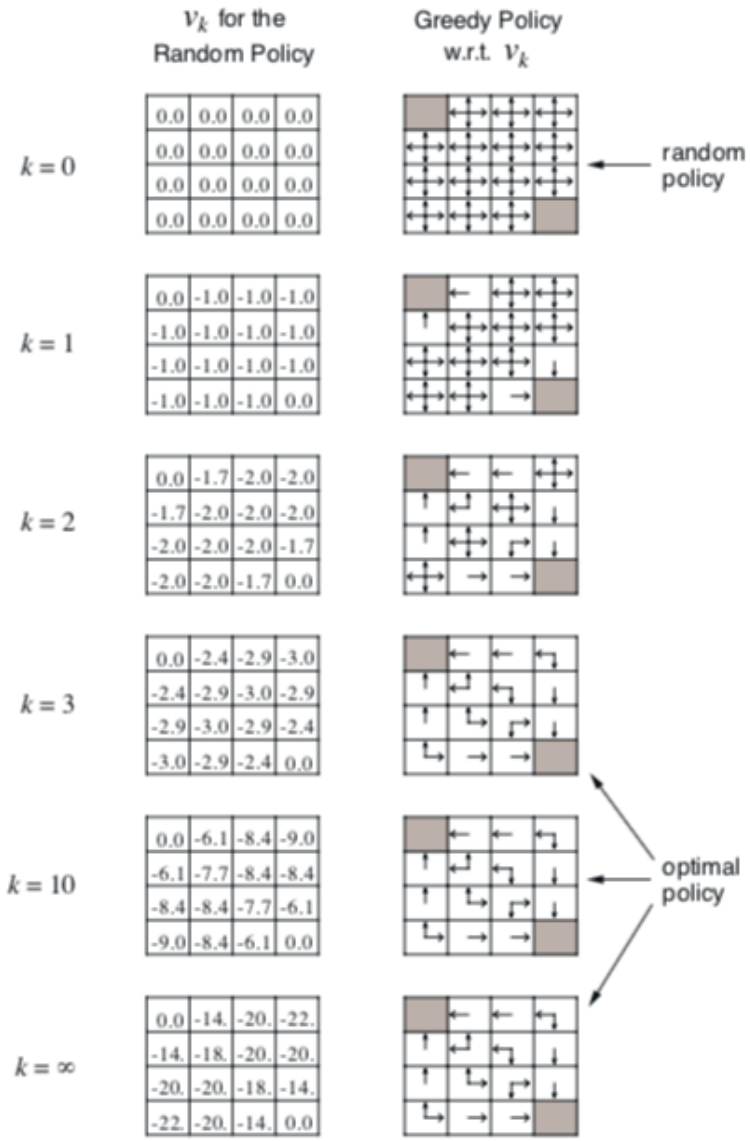
- Sweeping through the states in order: Gauss-Seidel value iteration

- Can perform updates to a smaller number of states. Still the algorithm converges as long as a state is not starved.
- In essence backup with priority states whose successors change the most. Avoid backing up a state if the successors are not changing.
- After a backup update the priority queue.

Example: 4*4 Grid World MDP

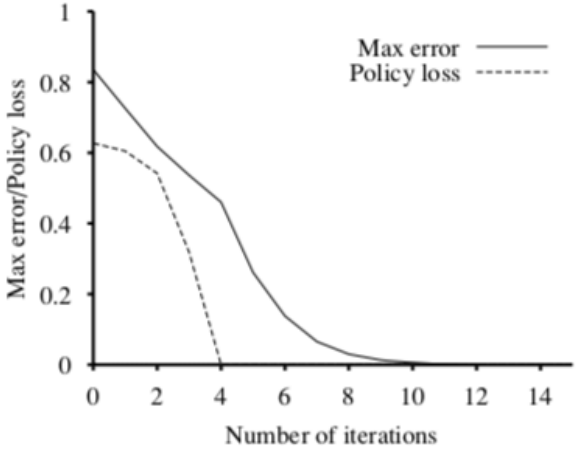
The extracted policy visualized at each stage.





The optimal policy converges before the values converge.

The optimal policy can be obtained even when the utility function estimate is inaccurate.



The maximum error of the utility estimates and the policy loss as a function of the number of iterations of value iteration.

Policy Evaluation

Here the agent is not choosing actions freely.
It is committed to policy π.

- An approach is iterative policy evaluation.
- Calculate the utilities (values) if the agent follows a given fixed policy until convergence.
- The computed value function may not be the optimal. It is the "best" we can get with a given policy.

Algorithm 1 Computing the value of an MDP policy.

POLICY-EVALUATION$(\mathcal{S}, \pi, R, T, \gamma, \epsilon)$

```
t = 0
for each state s ∈ S
    V_0[s] = 0
repeat
    change = 0
    t = t + 1
    for each state s ∈ S
        V_t[s] = (Σ_{s'} T(s, π[s], s') [R(s, π[s], s') + γ · V_{t-1}(s')])
        change = max(change, V_t[s] − V_{t-1}[s])
until change < ε
return V_t
```

Given the policy what is the value of each state when we are using this policy?

We average over actions because:

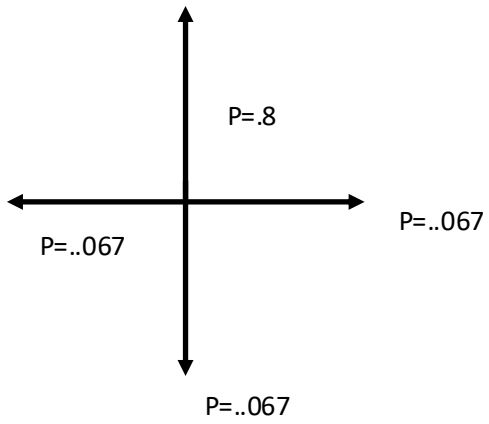
The agent is following π, which may be stochastic.

So this equation answers:
"If I behave like π forever, how much reward do I get?"

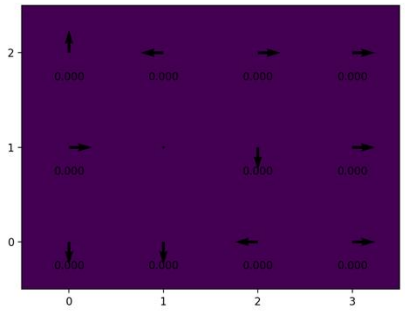
Policy Evaluation: Example

- Recall the grid world. Let us guess a policy at random, and evaluate it.
- The MDP model:
  - $S = \{(0,0),(0,1),(0,2),(1,1)\ldots(3,2)\}$
  - $T(s,a,s') = .8$ if s' is the nominal destination given s and a
  - $T(s,a,s') = .2/n$ if s' is a neighbour of the nominal destination given s and a and is a legal state (for n legal states)
  - $R(s) = +1$ if $s = (3,2)$
  - $R(s) = -1$ if $s = (3,1)$
  - $R(s) = -.1$ otherwise
  - $\gamma = 0.9$

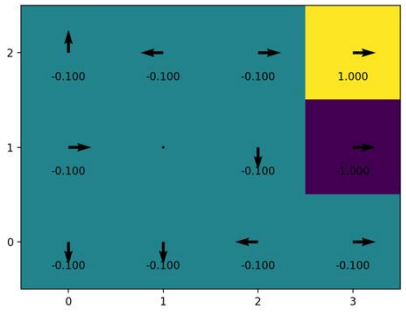
-.1	-.1	-.1	+1
-.1		-.1	-1
-.1	-.1	-.1	-.1



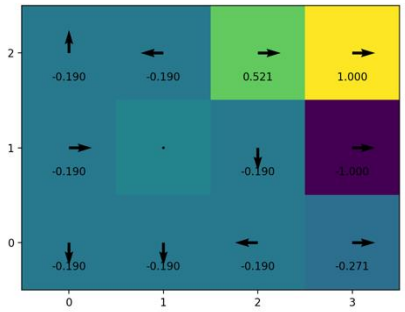
Policy Evaluation: Example



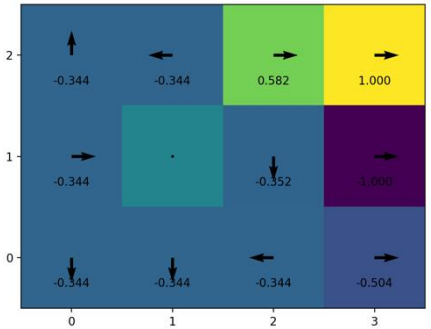
(a) V^0



(b) V^1



(c) V^2



(d) V^3

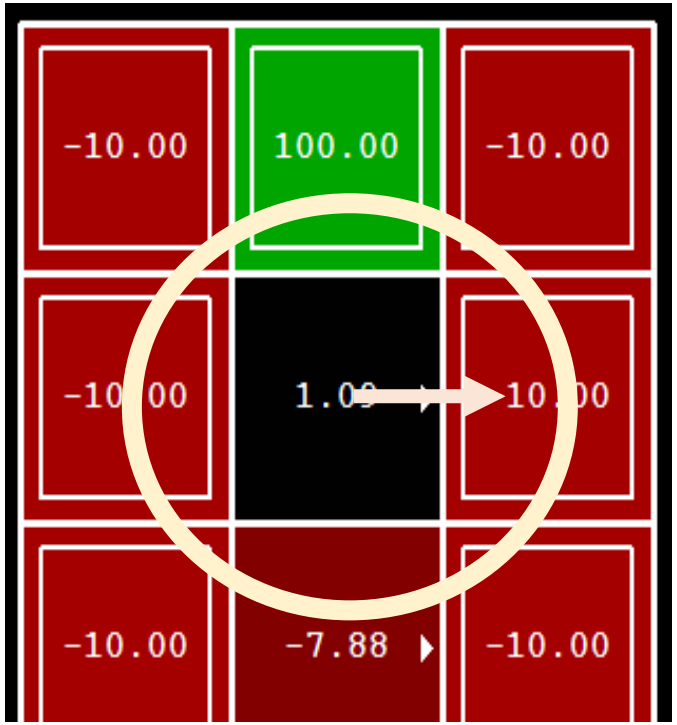
Let us consider the state $(2, 2)$

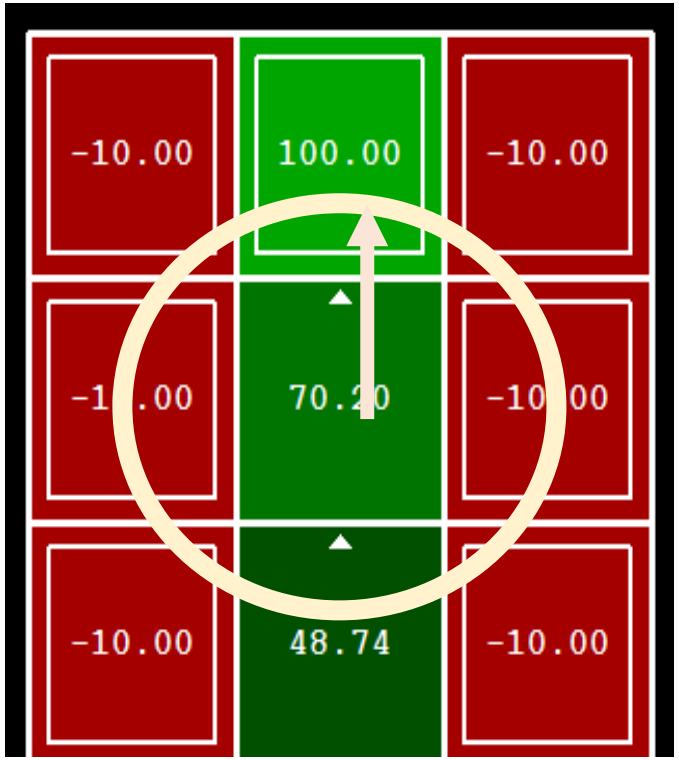
$$
\begin{aligned}
V^1(2,2) &= R((2,2)) + \gamma \sum_{s'} p(s'|s,a) V^0(s') \\
&= -.1 + .9(.8 \times 0 + .1 \times 0 + .1 \times 0) \\
&= -.1 \\
V^2(2,2) &= R((2,2)) + \gamma \sum_{s'} p(s'|s,a) V^1(s') \\
&= -.1 + .9(.8 \times 1 + .1 \times -.1 + .1 \times -1) \\
&= .521
\end{aligned}
$$

This was the evaluation of a random policy. Next, how can we improve the policy?

Policy Improvement

Illustration in a grid world example.





Policy Iteration

- Consider improving the policy
  - Given the fixed utility values for states (obtained via policy evaluation)
  - Examine if there is a better policy using one step look ahead.

Algorithm 2 An algorithm for computing an MDP policy by repeated evaluation and improvement.

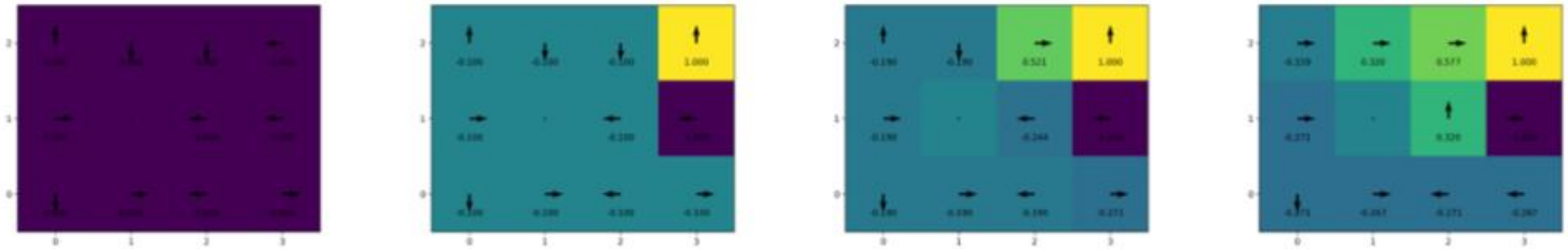
```
POLICY-ITERATION(S, A, R, T, γ, ε)
for each state s ∈ S
    i = rand(0, |A|)
    π[s] = a_i
t = 0
V_0 = POLICY-EVALUATION(S, π, R, T, γ, ε)
repeat
    change = 0
    t = t + 1
    for each state s ∈ S
        π_t[s] = argmax_a (Σ_{s'} T(s, a, s') [R(s, a, s') + γ · V_{t-1}(s')])
    V_t = POLICY-EVALUATION(S, π, R, T, γ, ε)
    for each state s ∈ S
        change = change + V_t[s] − V_{t-1}[s]
until change < ε
```

Key Idea:
- Evaluate and improve the policy.
- Interleave evaluation and improvement

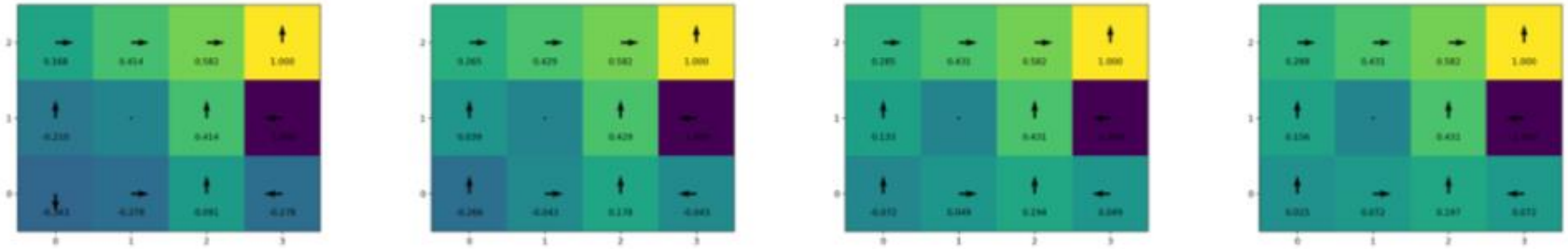
Policy Iteration Example

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*,$$

- Interleaving policy evaluation and policy improvement.
- Notice the policy changing over time.
- We are guaranteed to reach the optimal policy.



(e) (f) (g) (h)



(i) (j) (k) (l)

Value Iteration vs. Policy Iteration

- Value Iteration and Policy Iteration both compute the optimal values.
- Policy iteration may in many cases by more computationally expensive.
- For a finite policy space (i.e., discrete state, discrete actions), policy iteration is guaranteed to converge in a finite number of iterations.
  - Value iteration does not have the same guarantees.

Main Takeaway

- We now have a formal representation for what we informally described as an agent
  - An MDP represents the agent-environment interaction.
  - Given: States, Actions, Rewards, Discounting, Transition Function
  - Allows determination of the optimal policy.
    - Implication: For any autonomous system doing a task, write its MDP and simply solve for the policy (two algorithms available for us).
- Key Assumptions
  - Transition function is known – We know (analytically or via a probability distribution) the possible consequences of actions.
  - We know what reward we will get at any state/state transition in the MDP. Essentially, we know where the good states are.
  - **Under these assumptions we can compute the optimal behaviour** for the autonomous system.

But, what happens when action consequences and reward function is not known ahead of time (or is too complex) to even write? We will understand this in the next topic.